



Namal University, Mianwali
DEPARTMENT OF COMPUTER SCIENCES

FareShare: Ride-Sharing System

Database Design Document

Submitted By:

Muhammad Naveed	NUM-BSCS-2024-54
Munawar Ali	NUM-BSCS-2024-60
Areeba Tahir	NUM-BSCS-2024-15

Submission Date:

15th April, 2026

Supervisor: Ms. Asiya Batool

Revision History

Date	Version	Description of Changes	Author(s)
10 April, 2026	1.0	Initial draft	Naveed, Mu- nawar, Areeba
12 April, 2026	1.1	Added relationships and cardinalities	Naveed, Mu- nawar
15 April, 2026	2.0	Final version with Crow's Foot notation ERD	All

Contents

1	Project Overview	2
1.1	Introduction	2
1.2	Problem Statement	2
1.3	Project Objectives	2
1.4	GitHub Repository Link	2
2	Detailed Database Design	3
2.1	Entities	3
2.2	EERD Supertype/Subtype Structure	3
2.3	Data Dictionary	4
2.4	Business Rules	9
2.5	Relationships with Cardinalities	9
2.6	Crow's Foot Notation Legend	10
2.7	Entity Relationship Diagram (Crow's Foot Notation)	10
2.8	Data Integrity Constraints	12
3	References	13
4	Appendix: AI Usage and Prompts	14

1 Project Overview

1.1 Introduction

FareShare is an innovative ride-sharing and fare-sharing platform designed specifically for the Pakistani market, focusing initially on the Mianwali region. This document outlines the detailed database design for the system, which serves as the backbone for managing users (Riders and Drivers), ride bookings, dynamic fare-splitting, safety features, and administrative oversight.

The system operates on a cash-only model and requires manual driver verification by administrators, presenting unique challenges in data consistency, relational integrity, and audit logging. The database is engineered to handle complex, real-time relationships between multiple entities, such as multiple riders sharing a single ride resource, ensuring efficient and normalized data management.

1.2 Problem Statement

Current ride-hailing solutions are not fully adapted to the socio-economic and trust-based environment of smaller Pakistani cities. They lack a robust mechanism for ad-hoc fare-sharing among strangers or acquaintances, which is a critical need for cost-sensitive commuters like students and daily-wage workers.

From a database perspective, this creates a significant challenge in managing a many-to-many relationship between riders and a single ride. Furthermore, a purely digital payment and verification model is not feasible. The system must rely on a cash transaction log and a manual document verification process for drivers. The core database problem is modeling these complex interactions to ensure data integrity, prevent anomalies, and support features like dynamic fare splitting and emergency contact notifications in a way that is both reliable and fully auditable.

1.3 Project Objectives

General Objective: To design and implement a robust relational database that perfectly models the FareShare system's entities, their attributes, and their complex interconnections to ensure data integrity and support all functional requirements.

Specific Objectives:

- Design a conceptual model (EERD) accurately representing the supertype/subtype relationship of the User entity.
- Model the complex many-to-many relationship between Riders and Rides when fare-sharing is enabled.
- Ensure data integrity for the fare-splitting algorithm by accurately storing individual segment distances and fares.
- Create an auditable structure for driver document verification and SOS emergency alerts.

1.4 GitHub Repository Link

<https://github.com/Naveed83067/FareShare-Ride-Sharing-System/tree/main/DM%20Milestone%202>

2 Detailed Database Design

2.1 Entities

The following table identifies and defines the core entities central to the FareShare database system.

Sr. No	Entity Name	Description
1	User	A supertype entity representing any individual with a registered account.
2	Rider	Subtype of User - searches for, books, and pays for rides.
3	Driver	Subtype of User - owns vehicle, offers rides, earns money.
4	Admin	Subtype of User - manages system, verifies drivers.
5	Vehicle	Physical vehicle owned by a Driver.
6	Ride	Core transactional entity representing a journey.
7	RideParticipant	Associative entity resolving many-to-many relationship between Rider and Ride.
8	Payment	Cash payment transaction record.
9	DriverVerification	Tracks manual verification of driver documents by Admin.
10	SOSAlert	Emergency alert triggered during active ride.
11	EmergencyContact	Weak entity storing trusted contacts (max 3 per user).
12	Rating	Stores rating and feedback between users (max 2 per ride).

2.2 EERD Supertype/Subtype Structure

The `User` entity is modeled as a supertype with three subtypes: `Rider`, `Driver`, and `Admin`. This structure follows:

- **Disjoint constraint:** A user can have only one role. A single user account cannot be both a Rider and a Driver, nor can they hold multiple roles simultaneously. Represented by circle with (d).
- **Total constraint:** Every user must belong to exactly one subtype. When a user registers, they must select a role (Rider, Driver, or Admin). A user account without a role cannot access any system functionality, as the `user_role` discriminator attribute is mandatory (NOT NULL). Represented by **double line** from User to the specialization circle.

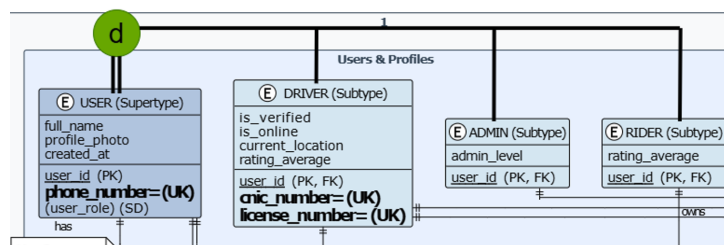


Figure 1: User Supertype with Subtypes (EERD Specialization) - Disjoint (d), Total (Double Line)

The discriminator attribute `user_role` in the `User` table determines which subtype a user

belongs to:

- `user_role = 'Rider'` → Corresponding record exists in `Rider` table
- `user_role = 'Driver'` → Corresponding record exists in `Driver` table
- `user_role = 'Admin'` → Corresponding record exists in `Admin` table

2.3 Data Dictionary

This section provides detailed attribute definitions for each entity.

2.3.1 User (Supertype)

Sr. No	Attribute	Data Type	Constraint	Description
1	<code>user_id</code>	UUID	PRIMARY KEY, NOT NULL	Unique identifier for every user
2	<code>phone_number</code>	VARCHAR(13)	NOT NULL, UNIQUE	Pakistan mobile number (+92 format)
3	<code>full_name</code>	VARCHAR(100)	NOT NULL	Full name of the user
4	<code>user_role</code>	ENUM('Rider','Driver','Admin')	NOT NULL	Subtype discriminator for user role
5	<code>profile_photo</code>	VARCHAR(255)	NULL	URL to profile picture
6	<code>created_at</code>	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Account creation timestamp

2.3.2 Rider (Subtype)

Sr. No	Attribute	Data Type	Constraint	Description
1	<code>user_id</code>	UUID	PRIMARY KEY, FOREIGN KEY	References User supertype
2	<code>rating_average</code>	DECIMAL(3,2)	DEFAULT 5.00	Average rating received from drivers

2.3.3 Driver (Subtype)

Sr. No	Attribute	Data Type	Constraint	Description
1	<code>user_id</code>	UUID	PRIMARY KEY, FOREIGN KEY	References User supertype
2	<code>cnic_number</code>	VARCHAR(15)	NOT NULL, UNIQUE	CNIC number (Computerized National Identity Card)

3	license_number	VARCHAR(20)	NOT NULL, UNIQUE	Valid driving license number
4	is_verified	BOOLEAN	NOT NULL, DEFAULT FALSE	Document verification status by Admin
5	is_online	BOOLEAN	NOT NULL, DEFAULT FALSE	Current availability status for ride requests
6	current_location	POINT	NULL	Last known GPS coordinates
7	rating_average	DECIMAL(3,2)	DEFAULT 5.00	Average rating received from riders

2.3.4 Admin (Subtype)

Sr. No	Attribute	Data Type	Constraint	Description
1	user_id	UUID	PRIMARY KEY, FOREIGN KEY	References User supertype
2	admin_level	ENUM('Junior','Senior')	NOT NULL	Authorization level for role-based access

2.3.5 Vehicle

Sr. No	Attribute	Data Type	Constraint	Description
1	vehicle_id	UUID	PRIMARY KEY, NOT NULL	Unique vehicle identifier
2	driver_id	UUID	FOREIGN KEY, NOT NULL, UNIQUE	One-to-one relationship with Driver
3	plate_number	VARCHAR(10)	NOT NULL, UNIQUE	Vehicle registration plate number
4	vehicle_type	ENUM('Bike','Rickshaw','Mini','Sedan')	NOT NULL	Vehicle type for pricing matrix
5	model	VARCHAR(50)	NULL	Make and model of the vehicle

2.3.6 Ride

Sr. No	Attribute	Data Type	Constraint	Description
1	ride_id	UUID	PRIMARY KEY, NOT NULL	Unique ride identifier

2	driver_id	UUID	FOREIGN KEY, NOT NULL	Driver who accepted and completed the ride
3	vehicle_id	UUID	FOREIGN KEY, NOT NULL	Vehicle used for the ride
4	status	ENUM('Requested','Accepted','In-Progress','Completed','Cancelled')	NOT NULL	Current lifecycle status of the ride
5	is_shared	BOOLEAN	NOT NULL, DEFAULT FALSE	Indicates if fare-sharing is enabled
6	total_fare	DECIMAL(10,2)	NOT NULL	Total calculated fare for the ride
7	created_at	TIMESTAMP	NOT NULL	Ride request timestamp
8	completed_at	TIMESTAMP	NULL	Ride completion timestamp

2.3.7 RideParticipant (Associative Entity)

Sr. No	Attribute	Data Type	Constraint	Description
1	participant_id	UUID	PRIMARY KEY, NOT NULL	Unique participant record ID
2	ride_id	UUID	FOREIGN KEY, NOT NULL	References the Ride entity
3	rider_id	UUID	FOREIGN KEY, NOT NULL	References the Rider entity
4	pickup_location	POINT	NOT NULL	Rider's specific pickup GPS coordinates
5	dropoff_location	POINT	NOT NULL	Rider's specific dropoff GPS coordinates
6	segment_fare	DECIMAL(10,2)	NOT NULL	Individual rider's share of the total fare
7	is_primary_rider	BOOLEAN	NOT NULL, DEFAULT FALSE	Indicates if this rider booked the ride initially

2.3.8 Payment

Sr. No	Attribute	Data Type	Constraint	Description
1	payment_id	UUID	PRIMARY KEY, NOT NULL	Unique payment identifier
2	participant_id	UUID	FOREIGN KEY, NOT NULL, UNIQUE	One-to-one relationship with RideParticipant
3	amount	DECIMAL(10,2)	NOT NULL	Cash amount received (equals segment_fare)

4	is_received	BOOLEAN	NOT NULL, DE-FAULT FALSE	Payment confirmation flag
5	paid_at	TIMESTAMP	NULL	Timestamp when payment was confirmed

2.3.9 DriverVerification

Sr. No	Attribute	Data Type	Constraint	Description
1	verification_id	UUID	PRIMARY KEY, NOT NULL	Unique verification log ID
2	driver_id	UUID	FOREIGN KEY, NOT NULL	Driver being verified
3	admin_id	UUID	FOREIGN KEY, NOT NULL	Admin who performed the verification
4	verification_status	ENUM('Approved','Rejected','Pending')	NOT NULL	Outcome of the document review
5	comments	TEXT	NULL	Admin comments or rejection reason
6	verified_at	TIMESTAMP	NOT NULL	Timestamp of verification action

2.3.10 SOSAlert

Sr. No	Attribute	Data Type	Constraint	Description
1	alert_id	UUID	PRIMARY KEY, NOT NULL	Unique alert identifier
2	ride_id	UUID	FOREIGN KEY, NOT NULL	Ride during which alert was triggered
3	triggered_by	UUID	FOREIGN KEY, NOT NULL	User (Rider or Driver) who triggered alert
4	alert_status	ENUM('Active','Responding','Resolved')	NOT NULL	Current emergency response status
5	location	POINT	NOT NULL	GPS coordinates where alert was triggered
6	triggered_at	TIMESTAMP	NOT NULL	Timestamp when alert was triggered

2.3.11 EmergencyContact (Weak Entity)

Sr. No	Attribute	Data Type	Constraint	Description
1	contact_id	UUID	PRIMARY KEY, NOT NULL	Unique contact identifier
2	user_id	UUID	FOREIGN KEY, NOT NULL	Owning user (part of weak entity dependency)
3	contact_name	VARCHAR(100)	NOT NULL	Full name of emergency contact person
4	phone_number	VARCHAR(13)	NOT NULL	Phone number of emergency contact
5	relationship	VARCHAR(50)	NULL	Relationship to the user (e.g., Father, Mother)

2.3.12 Rating

Sr. No	Attribute	Data Type	Constraint	Description
1	rating_id	UUID	PRIMARY KEY, NOT NULL	Unique rating identifier
2	ride_id	UUID	FOREIGN KEY, NOT NULL	Ride being rated
3	rater_user_id	UUID	FOREIGN KEY, NOT NULL	User (Rider or Driver) giving the rating
4	rated_user_id	UUID	FOREIGN KEY, NOT NULL	User (Driver or Rider) receiving the rating
5	score	TINYINT(1)	NOT NULL, CHECK (score BETWEEN 1 AND 5)	Star rating from 1 to 5
6	comment	TEXT	NULL	Optional textual feedback
7	created_at	TIMESTAMP	NOT NULL	Timestamp when rating was submitted

2.4 Business Rules

The following business rules govern data integrity and relationships within the FareShare system:

1. A `User` must have exactly one role: `'Rider'`, `'Driver'`, or `'Admin'` (disjoint specialization).
2. A `Driver` must possess exactly one registered `Vehicle` (1:1 relationship).
3. A `Driver` must be verified (`is_verified = TRUE`) before they can go online (`is_online = TRUE`) and accept rides.
4. A `Ride` can be regular (`is_shared = FALSE`) with exactly one `Rider`, or shared (`is_shared = TRUE`) with multiple `Riders`.
5. For a shared ride, the `RideParticipant` entity must record each `Rider`'s specific `segment_fare`.
6. A `Payment` record is created for each `RideParticipant` (1:1 relationship) and marked `is_received = TRUE` only upon driver's cash confirmation.
7. An `SOSAlert` can only be triggered by a `Rider` or `Driver` who is an active participant in an `'In-Progress'` ride.
8. An `Admin` must perform manual verification to change a `DriverVerification` record and the corresponding `Driver`'s `is_verified` flag.
9. Maximum **3 emergency contacts** per user.
10. Maximum **2 ratings per ride** (one from rider to driver, one from driver to rider).
11. Total fare for a ride must equal the sum of all `segment_fares` (application-level enforcement).

2.5 Relationships with Cardinalities

The following table defines all relationships between entities with their types and cardinalities using Crow's Foot notation.

#	Entity A	Entity B	Relationship Type	Min	Max
1	User	Rider	Specialization (Disjoint, Total)	0	1
2	User	Driver	Specialization (Disjoint, Total)	0	1
3	User	Admin	Specialization (Disjoint, Total)	0	1
4	Driver	Vehicle	One-to-One	1	1
5	Driver	Ride	One-to-Many	0	*
6	Vehicle	Ride	One-to-Many	0	*
7	Rider	RideParticipant	One-to-Many	1	*
8	Ride	RideParticipant	One-to-Many	1	*
9	RideParticipant	Payment	One-to-One	1	1
10	Admin	DriverVerification	One-to-Many	0	*

11	Driver	DriverVerification	One-to-Many	1	*
12	Ride	SOSAlert	One-to-Zero-or-One	0	1
13	User	SOSAlert	One-to-Many	0	*
14	User	EmergencyContact	One-to-Many	0	3
15	Ride	Rating	One-to-Many	0	2
16	User	Rating	One-to-Many	0	*

Note: For specialization rows (1-3), each User must belong to exactly ONE subtype (Rider, Driver, or Admin) due to the Total constraint. The (0,1) cardinality per row reflects that from the subtype's perspective, a specific subtype record may not exist for every User.

2.6 Crow's Foot Notation Legend

The ERD uses Crow's Foot notation with the following symbols:

Symbol	Meaning	Description
—	Exactly one	Mandatory single relationship
—o	Zero or one	Optional single relationship
—{	Many (zero or many)	Optional multiple relationship
— {	One or many	Mandatory multiple relationship

Additional Notation:

- **PK** = Primary Key
- **FK** = Foreign Key
- **UK** = Unique Key
- **ISA** = Supertype/Subtype relationship
- **(d)** = Disjoint constraint (user can have only one role)
- **Double line** = Total specialization (every user must have a role)

2.7 Entity Relationship Diagram (Crow's Foot Notation)

The following ERD uses Crow's Foot notation to represent the logical structure of the FareShare database. Rectangles represent entities, diamonds represent relationships, and cardinalities are shown using the Crow's Foot symbols at the ends of relationship lines.

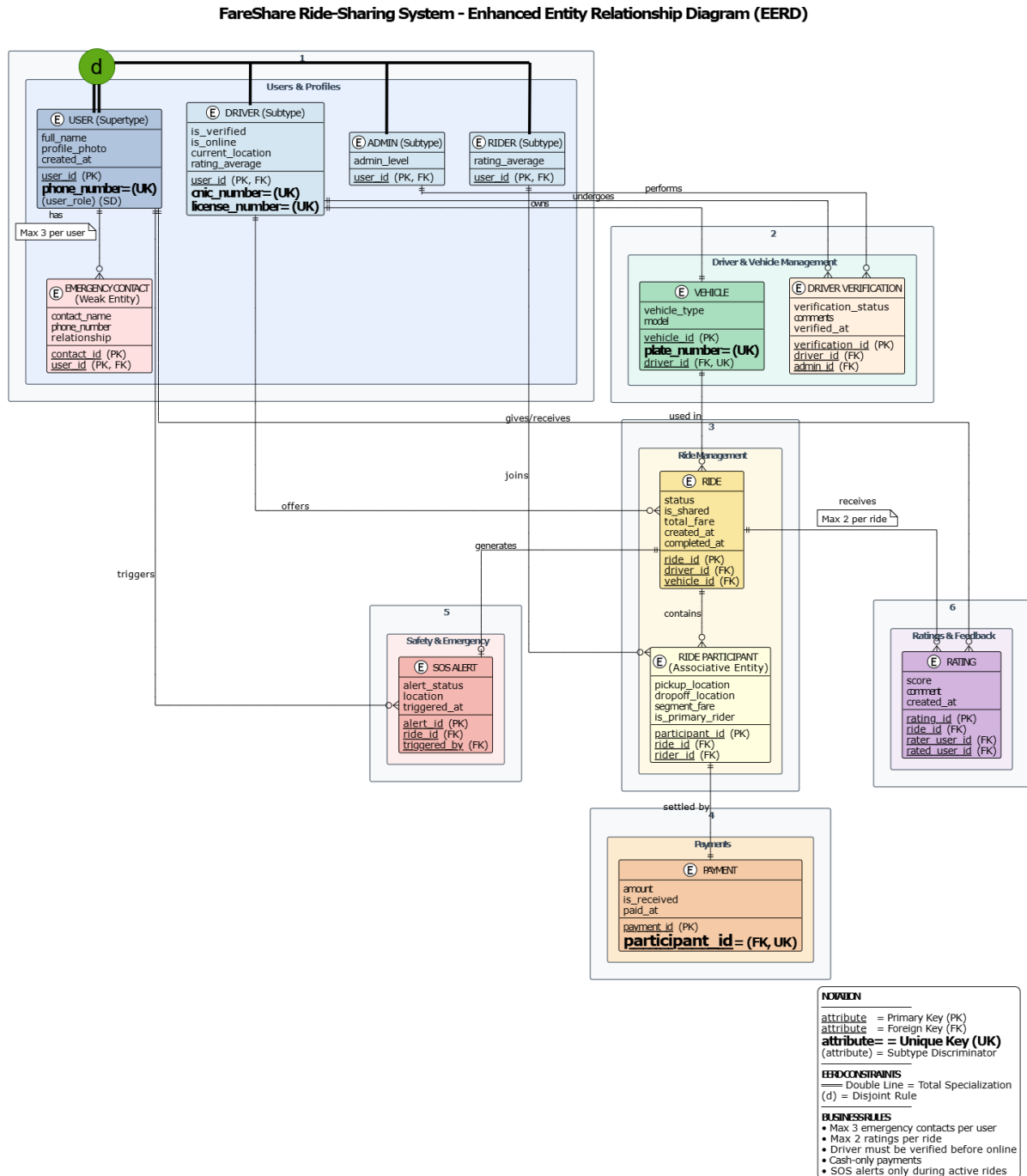


Figure 2: FareShare Entity Relationship Diagram in Crow's Foot Notation

ERD Description:

- **User Specialization:** User (supertype) has an ISA relationship with Rider, Driver, and Admin (subtypes) - Disjoint (d), Total (double line)
- **Driver Relationships:** Driver (1) - owns - (1) Vehicle; Driver (1) - offers - (*) Ride
- **Vehicle Relationships:** Vehicle (1) - used in - (*) Ride

- **Ride Relationships:** Ride (1) - contains - (*) RideParticipant; Ride (1) - generates - (0,1) SOSAlert
- **Rider Relationships:** Rider (1) - joins - (*) RideParticipant (resolves M:N relationship between Rider and Ride)
- **Payment Relationship:** RideParticipant (1) - settled by - (1) Payment
- **Verification Relationships:** Admin (1) - performs - (*) DriverVerification; Driver (1) - undergoes - (*) DriverVerification
- **Safety Relationships:** User (1) - triggers - (*) SOSAlert; User (1) - has - (0,3) EmergencyContact
- **Rating Relationships:** Ride (1) - receives - (0,2) Rating; User (1) - gives/receives - (*) Rating

2.8 Data Integrity Constraints

ID	Rule Description	Enforcement
DI-01	Driver cannot go online without <code>is_verified = TRUE</code>	Application logic
DI-02	Total fare must equal <code>SUM(segment_fare)</code> for shared rides	Application/Trigger
DI-03	SOSAlert can only link to 'In-Progress' rides	Application validation
DI-04	Rating score must be between 1 and 5	CHECK constraint
DI-05	A user cannot rate the same ride twice	UNIQUE constraint on (ride_id, rater_user_id)
DI-06	Vehicle must exist before a Ride can use it	FOREIGN KEY constraint
DI-07	Emergency contacts limited to 3 per user	Application validation
DI-08	Payment amount must equal <code>RideParticipant.segment_fare</code>	Application validation
DI-09	Max 2 ratings per ride	Application validation

3 References

1. FareShare Software Requirements Specification (SRS), Version 2.0, Team FareShare, January 11, 2026.
2. Project Proposal: FareShare Database System, Team FareShare, March 12, 2026.
3. Hoffer, J. A., Ramesh, V., & Topi, H. (2019). *Modern Database Management* (13th ed.). Pearson.
4. Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson.
5. IEEE Std 830-1984. (1984). *IEEE Guide to Software Requirements Specifications*. IEEE Computer Society.

4 Appendix: AI Usage and Prompts

AI Tools Used:

- Google Gemini - For generating LaTeX boilerplate and ERD structure
- DeepSeek AI - For enhancing database design and relationship modeling

Purpose: To generate LaTeX boilerplate, Crow's Foot notation ERD structure based on project SRS, and to enhance the database design document according to milestone requirements.

Prompts Used:

Prompt	Purpose	Response Summary
"List all entities for a ride-sharing system with fare splitting in Pakistan"	Identify core entities	Received 12 entities including User supertype with Rider/Driver/Admin subtypes
"Generate Crow's Foot notation ERD with cardinalities for FareShare system"	Create milestone-compliant ERD	Produced Crow's Foot structure with cardinalities and relationship lines
"Convert the data dictionary into colored LaTeX tables"	Format tables professionally	Generated colored tables with header styling and proper column widths
"Write business rules for cash-based ride sharing with manual driver verification"	Define constraints	Created comprehensive business rules covering all entity interactions
"Create relationships table with cardinalities in Crow's Foot format"	Document entity relationships	Generated complete relationship matrix with 16 relationship entries
"Add data integrity constraints and performance indexes"	Enhance database design	Created DI-01 to DI-09 constraints and performance indexes

Declaration: We declare that AI tools were used solely for understanding concepts, generating templates, and improving documentation quality. All database design decisions, entity relationships, and business rules were reviewed and validated by the team.